

Logic

The program implements a custom `my_atof()` function to convert a string into a floating-point number without using the standard library `atof()`.

The conversion is performed in the following sequence:

1. Skip leading whitespace

The function ignores spaces, tabs, newlines, carriage returns, form feeds, and vertical tabs before processing the number.

2. Check the sign

It checks whether the first meaningful character is '+' or '-'.

'+' → positive number

'-' → negative number

No sign → positive number by default

3. Convert the integer part

Each character from '0' to '9' before a decimal point is converted into its numeric value using:
`*s - '0'`

The result is accumulated using:

```
res = res * 10 + (*s - '0');
```

For example, "123" is processed as:

'1' → '12' → '123'

4. Process the fractional part

If a decimal point ('.') is encountered, the function begins processing the digits that follow. To maintain the correct mathematical placement, a 'power' variable (initialized at '1.0') is introduced.

For each digit after the decimal:

```
res = res * 10.0 + (*s - '0');
```

```
power = power * 10.0;
```

After all fractional digits are processed, the accumulated result is divided by 'power'.

For example, if the input is '.25', the loop makes 'res = 25' and 'power = 100'. Then, '25 / 100' correctly yields '0.25'.

5. Stop at the first invalid character

Conversion continues only while the current character is a valid digit (or a single decimal point).

If an invalid character is hit, parsing safely stops. For example:

'123abc' → '123.000000'

' +2+.2+53' → '2.000000' (stops parsing at the second '+')

6. Apply the sign and return

Finally, the calculated value is multiplied by the 'sign' and returned as a 'double'.

In simple terms:

String → Ignore spaces → Detect sign → Convert integer digits → Detect decimal → Convert fractional digits → Divide by power → Apply sign → Return double